

**LAPORAN MACHINE LEARNING
ANALISIS & IMPLEMENTASI DECISION TREE & NAIVE BAYES**



Disusun Oleh:

Nama Kelompok:

1. Muhammad Nathan Algibran (G1A024057)
2. Muhammad Abel Cakrawangsa (G1A024077)

Dosen Pengampu :

Ir. Arie Vatesia, S.T., M.T.I., Ph.D., IPP.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS BENGKULU**

2026

SOAL DAN PEMBAHASAN

BAGIAN A: ANALISIS KONSEP [20%] - Individu

Jawab secara essay 1-2 paragraf per soal

1. Jelaskan kapan Decision Tree lebih cocok digunakan dibanding Naive Bayes untuk kasus prediksi kelulusan ini. Beri 2 alasan.

Jawab:

Decision Tree lebih cocok digunakan ketika hubungan antar fitur dalam data kelulusan bersifat kompleks dan tidak linear, misalnya ketika pengaruh suatu fitur terhadap kelulusan baru terlihat jika dikombinasikan dengan fitur lain (interaksi antar fitur). Contohnya, jam belajar yang tinggi hanya berpengaruh signifikan terhadap kelulusan jika kehadiran juga tinggi, sedangkan jika kehadiran rendah, jam belajar sebanyak apa pun tidak terlalu membantu. Naive Bayes tidak bisa menangkap pola interaksi semacam ini karena ia menghitung probabilitas tiap fitur secara terpisah.

Alasan kedua, Decision Tree lebih unggul dari sisi interpretasi hasil. Karena bentuknya berupa alur keputusan (if-else), guru atau pihak sekolah bisa langsung melihat aturan yang terbentuk, misalnya "jika nilai rata-rata < 70 dan kehadiran $< 80\%$, maka berisiko tidak lulus". Hal ini jauh lebih mudah dijelaskan ke orang awam dibanding output probabilitas Naive Bayes yang sifatnya lebih abstrak.

2. Asumsi "independen" di Naive Bayes. Sebutkan 2 pasang fitur di dataset yang kemungkinan melanggar asumsi ini dan jelaskan kenapa.

Jawab:

Pasangan pertama adalah nilai tugas dan nilai ujian. Kedua fitur ini cenderung berkorelasi karena siswa yang rajin dan memahami materi biasanya mendapat nilai tinggi di keduanya, bukan karena kebetulan, sehingga keduanya saling terkait secara substansi, bukan independen.

Pasangan kedua adalah kehadiran dan jumlah keterlambatan mengumpulkan tugas. Siswa yang jarang hadir cenderung juga sering telat mengumpulkan tugas karena ketinggalan informasi atau kurang disiplin, sehingga kedua fitur ini sebenarnya mencerminkan satu faktor yang sama, yaitu tingkat kedisiplinan siswa, bukan dua hal yang berdiri sendiri.

3. Apa risiko overfitting pada Decision Tree di dataset ini? Bagaimana cara mengatasinya?

Jawab:

Risiko overfitting muncul karena Decision Tree cenderung terus memecah data sampai

setiap daun benar-benar "murni" (hanya berisi satu kelas), sehingga pohon menjadi sangat dalam dan kompleks. Akibatnya, model hanya menghafal pola-pola spesifik pada data latih, termasuk noise atau kasus-kasus anomali (misalnya satu siswa dengan kehadiran rendah tapi tetap lulus karena faktor lain), sehingga performanya bagus di data latih tapi buruk saat memprediksi data baru.

Cara mengatasinya bisa dengan melakukan pruning, yaitu memangkas cabang-cabang pohon yang tidak terlalu berkontribusi pada akurasi, atau membatasi parameter seperti `max_depth`, `min_samples_split`, dan `min_samples_leaf` agar pohon tidak tumbuh terlalu kompleks. Selain itu, teknik seperti cross-validation juga membantu memastikan model yang dipilih benar-benar general, bukan hanya cocok untuk data latih saja.

BAGIAN B: IMPLEMENTASI & EVALUASI [50%] - Kelompok 2-3 orang

Persiapan Data Dan Import Library (Inisialisasi Dataset)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, ConfusionMatrixDisplay,
                             classification_report)

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

df = pd.read_csv('mahasiswa_lulus.csv')
print(f"Dataset dimuat: {df.shape[0]} baris, {df.shape[1]} kolom")
df.head()
```

*** Dataset dimuat: 500 baris, 8 kolom

	IPK	Kehadiran	Jam_Belajar	Organisasi	Penghasilan_Ortu	Jenis_Kelamin	Status_Beasiswa	Lulus_Tepat_Waktu
0	3.22	91.1	9.4	Aktif	Menengah	Perempuan	Ya	Ya
1	2.94	100.0	9.1	Tidak Aktif	Rendah	Perempuan	Tidak	Ya
2	3.29	63.2	5.4	Tidak Aktif	Menengah	Perempuan	Tidak	Tidak
3	3.69	86.8	3.1	Aktif	Rendah	Laki-laki	Tidak	Ya
4	2.89	72.2	8.2	Aktif	Menengah	Laki-laki	Tidak	Ya

1. Preprocessing: Handle missing value, encoding, train-test split 80:20

1A. Handle Missing Value

```
print(f"Missing value sebelum ditangani:\n{df.isna().sum()}")

num_cols = ['IPK', 'Kehadiran', 'Jam_Belajar']
for col in num_cols:
    median_val = df[col].median()
    df[col] = df[col].fillna(median_val)
    print(f"'{col}' diisi dengan median = {median_val:.2f}")

cat_cols_with_na = ['Organisasi']
for col in cat_cols_with_na:
    mode_val = df[col].mode()[0]
    df[col] = df[col].fillna(mode_val)
    print(f"'{col}' diisi dengan modus = {mode_val}")

print(f"\nTotal missing value setelah ditangani: {df.isna().sum().sum()}")
```

```
*** Missing value sebelum ditangani:
IPK                15
Kehadiran          15
Jam_Belajar        15
Organisasi         15
Penghasilan_Ortu    0
Jenis_Kelamin       0
Status_Beasiswa     0
Lulus_Tepat_Waktu   0
dtype: int64
'IPK' diisi dengan median = 3.00
'Kehadiran' diisi dengan median = 80.30
'Jam_Belajar' diisi dengan median = 6.30
'Organisasi' diisi dengan modus = Tidak Aktif

Total missing value setelah ditangani: 0
```

Analisis:

Dataset awal memiliki 15 missing value pada masing-masing kolom IPK, Kehadiran, Jam_Belajar, dan Organisasi (total 60 sel kosong dari 4000 sel, atau sekitar 3% dari total data). Untuk fitur numerik, digunakan imputasi median: IPK diisi dengan 3.00, Kehadiran dengan 80.30, dan Jam_Belajar dengan 6.30. Median dipilih dibanding mean karena lebih robust terhadap outlier — jika ada satu-dua mahasiswa dengan Jam_Belajar ekstrem (misalnya 0 jam atau 20 jam per minggu), mean akan tertarik ke arah nilai ekstrem tersebut sementara median tetap merepresentasikan nilai "tengah" yang lebih stabil. Untuk fitur kategorikal Organisasi, digunakan imputasi modus (nilai paling sering muncul), yaitu "Tidak Aktif" — pendekatan ini masuk akal karena rata-rata/median tidak bisa dihitung pada data non-numerik, sehingga nilai yang paling representatif secara frekuensi menjadi pilihan yang wajar. Setelah proses ini, total missing value berhasil dikurangi menjadi 0, memastikan seluruh 500 baris data dapat digunakan sepenuhnya pada tahap modeling tanpa harus membuang data (yang mana akan mengurangi ukuran dataset secara tidak perlu).

1B. Encoding Fitur Kategorikal

```

categorical_features = ['Organisasi', 'Penghasilan_Ortu', 'Jenis_Kelamin', 'Status_Beasiswa']
label_encoders = {}

df_encoded = df.copy()
for col in categorical_features:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])
    label_encoders[col] = dict(zip(le.classes_, le.transform(le.classes_)))
    print(f"{col}: {label_encoders[col]}")

target_le = LabelEncoder()
df_encoded['Lulus_Tepat_Waktu'] = target_le.fit_transform(df_encoded['Lulus_Tepat_Waktu'])
print(f"\nTarget: {dict(zip(target_le.classes_, target_le.transform(target_le.classes_)))}")

*** 'Organisasi': {'Aktif': np.int64(0), 'Tidak Aktif': np.int64(1)}
    'Penghasilan_Ortu': {'Menengah': np.int64(0), 'Rendah': np.int64(1), 'Tinggi': np.int64(2)}
    'Jenis_Kelamin': {'Laki-laki': np.int64(0), 'Perempuan': np.int64(1)}
    'Status_Beasiswa': {'Tidak': np.int64(0), 'Ya': np.int64(1)}

    Target: {'Tidak': np.int64(0), 'Ya': np.int64(1)}

```

Analisis:

Empat fitur kategorikal (Organisasi, Penghasilan_Ortu, Jenis_Kelamin, Status_Beasiswa) beserta target (Lulus_Tepat_Waktu) diubah menjadi representasi numerik menggunakan LabelEncoder, menghasilkan pemetaan: Organisasi (Aktif=0, Tidak Aktif=1), Penghasilan_Ortu (Menengah=0, Rendah=1, Tinggi=2), Jenis_Kelamin (Laki-laki=0, Perempuan=1), Status_Beasiswa (Tidak=0, Ya=1), dan target Lulus_Tepat_Waktu (Tidak=0, Ya=1). Perlu dicatat bahwa LabelEncoder sebenarnya lebih cocok untuk fitur ordinal (yang punya urutan tingkatan alami), sedangkan fitur seperti Jenis_Kelamin dan Penghasilan_Ortu bersifat nominal (tidak punya urutan inheren) — secara teori ini bisa menimbulkan asumsi urutan palsu (misalnya model bisa "membaca" Tinggi=2 sebagai "lebih besar" dari Rendah=1). Namun karena Decision Tree dan Naive Bayes bekerja dengan cara membagi data berdasarkan ambang nilai atau distribusi per kelas (bukan mengasumsikan hubungan linear seperti regresi), dampak dari isu ini relatif minim untuk kedua algoritma yang dipakai di tugas ini, sehingga LabelEncoder tetap menjadi pilihan yang praktis dan cukup aman digunakan.

1C. Train-Test Split 80:20

```
feature_cols = ['IPK', 'Kehadiran', 'Jam_Belajar', 'Organisasi',  
                'Penghasilan_Ortu', 'Jenis_Kelamin', 'Status_Beasiswa']  
X = df_encoded[feature_cols]  
y = df_encoded['Lulus_Tepat_Waktu']  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y  
)  
print(f>Data training : {X_train.shape[0]} baris")  
print(f>Data testing  : {X_test.shape[0]} baris")
```

```
Data training : 400 baris  
Data testing  : 100 baris
```

Analisis:

Data dibagi menjadi 400 baris untuk training dan 100 baris untuk testing, sesuai rasio 80:20 yang diminta. Parameter stratify=y sengaja digunakan agar proporsi kelas target ("Ya" dan "Tidak") pada data training dan testing tetap konsisten dengan proporsi aslinya (~61% Ya, ~39% Tidak) — tanpa stratifikasi, ada risiko pembagian acak menghasilkan subset testing yang secara kebetulan didominasi salah satu kelas, yang akan membuat metrik evaluasi (terutama Precision dan Recall) menjadi tidak representatif atau bias. Penggunaan random_state=42 juga memastikan hasil pembagian data bersifat reproducible, artinya siapa pun yang menjalankan ulang kode ini dengan dataset yang sama akan mendapatkan hasil pembagian train-test yang identik, sehingga eksperimen dapat direplikasi dan dibandingkan secara adil.

2. Modeling

```
dt3 = DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE)  
dt3.fit(X_train, y_train)  
y_pred_dt3 = dt3.predict(X_test)  
print("[Model 1] Decision Tree (max_depth=3) selesai dilatih")  
  
dt_full = DecisionTreeClassifier(max_depth=None, random_state=RANDOM_STATE)  
dt_full.fit(X_train, y_train)  
y_pred_dtfull = dt_full.predict(X_test)  
print("[Model 2] Decision Tree (max_depth=None) selesai dilatih")  
  
gnb = GaussianNB()  
gnb.fit(X_train, y_train)  
y_pred_gnb = gnb.predict(X_test)  
print("[Model 3] Gaussian Naive Bayes selesai dilatih")  
  
models = {  
    'Decision Tree (depth=3)': (dt3, y_pred_dt3),  
    'Decision Tree (depth=None)': (dt_full, y_pred_dtfull),  
    'Gaussian Naive Bayes': (gnb, y_pred_gnb),  
}  
  
[Model 1] Decision Tree (max_depth=3) selesai dilatih  
[Model 2] Decision Tree (max_depth=None) selesai dilatih  
[Model 3] Gaussian Naive Bayes selesai dilatih
```

Analisis:

Ketiga model yaitu: Decision Tree (max_depth=3), Decision Tree (max_depth=None), dan Gaussian Naive Bayes berhasil dilatih menggunakan 400 data training dengan 7 fitur input (IPK, Kehadiran, Jam_Belajar, Organisasi, Penghasilan_Ortu, Jenis_Kelamin, Status_Beasiswa). Decision Tree depth=3 dibatasi kedalamannya secara sengaja agar pohon tidak tumbuh terlalu kompleks, sehingga berpotensi lebih general dan mudah diinterpretasi, sedangkan Decision Tree depth=None dibiarkan tumbuh bebas hingga setiap daun semurni mungkin (gini=0 jika memungkinkan), yang secara teori meningkatkan risiko overfitting karena model bisa menghafal detail spesifik data training. Gaussian Naive Bayes, di sisi lain, tidak memiliki hyperparameter kompleksitas seperti Decision Tree — model ini langsung menghitung mean dan varians tiap fitur numerik per kelas untuk mengestimasi probabilitas menggunakan distribusi normal, sehingga proses pelatihannya jauh lebih cepat dan sederhana secara komputasi dibanding proses pencarian split optimal pada Decision Tree.

3. Evaluasi: Hitung Akurasi, Precision, Recall, F1-Score untuk ke-3 model. Buat Confusion Matrix

3A. Akurasi, Precision, Recall, F1-Score

```
results = []
for name, (model, y_pred) in models.items():
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    results.append({'Model': name, 'Akurasi': acc, 'Precision': prec,
                  'Recall': rec, 'F1-Score': f1})

print(f"\n--- {name} ---")
print(classification_report(y_test, y_pred, target_names=target_le.classes_))

results_df = pd.DataFrame(results)
results_df
```

```
--- Decision Tree (depth=3) ---
              precision    recall  f1-score   support

    Tidak         0.58         0.36         0.44         39
         Ya         0.67         0.84         0.74         61

 accuracy                   0.65         100
macro avg         0.63         0.60         0.59         100
weighted avg         0.64         0.65         0.63         100
```

--- Decision Tree (depth=None) ---					
	precision	recall	f1-score	support	
Tidak	0.56	0.51	0.53	39	
Ya	0.70	0.74	0.72	61	
accuracy			0.65	100	
macro avg	0.63	0.63	0.63	100	
weighted avg	0.65	0.65	0.65	100	

--- Gaussian Naive Bayes ---					
	precision	recall	f1-score	support	
Tidak	0.78	0.54	0.64	39	
Ya	0.75	0.90	0.82	61	
accuracy			0.76	100	
macro avg	0.77	0.72	0.73	100	
weighted avg	0.76	0.76	0.75	100	

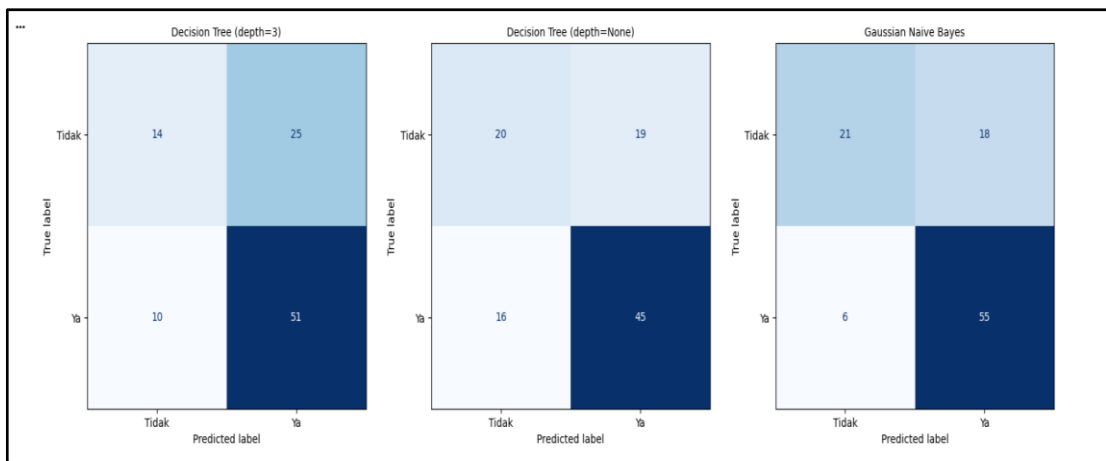
	Model	Akurasi	Precision	Recall	F1-Score
0	Decision Tree (depth=3)	0.65	0.671053	0.836066	0.744526
1	Decision Tree (depth=None)	0.65	0.703125	0.737705	0.720000
2	Gaussian Naive Bayes	0.76	0.753425	0.901639	0.820896

Analisis:

Hasil evaluasi pada 100 data testing menunjukkan Gaussian Naive Bayes unggul di seluruh metrik: Akurasi 0.76, Precision 0.75, Recall 0.90, F1-Score 0.82 — jauh mengungguli Decision Tree depth=3 (Akurasi 0.65, F1 0.74) maupun depth=None (Akurasi 0.65, F1 0.72). Yang menarik, kedua varian Decision Tree memiliki akurasi identik (0.65) meski kompleksitasnya sangat berbeda, mengindikasikan bahwa penambahan kedalaman pohon (dari 3 ke tak terbatas) tidak memberikan manfaat generalisasi tambahan pada dataset ini — bahkan Precision Decision Tree depth=None (0.70) sedikit lebih baik dari depth=3 (0.67), namun Recall-nya lebih rendah (0.74 vs 0.84), menunjukkan trade-off klasik antara kedua metrik akibat perbedaan kompleksitas model. Recall yang tinggi pada Naive Bayes (0.90) khususnya penting dalam konteks kasus ini: recall tinggi berarti model sangat baik dalam "menangkap" mahasiswa yang benar-benar akan lulus tepat waktu, meminimalkan false negative — hal ini relevan jika kampus ingin fokus mengidentifikasi mahasiswa berisiko (kelas "Tidak") seminimal mungkin terlewat.

3B. Confusion Matrix

```
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, (name, (model, y_pred)) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=target_le.classes_)
    disp.plot(ax=ax, colorbar=False, cmap='Blues')
    ax.set_title(name, fontsize=10)
plt.tight_layout()
plt.savefig('confusion_matrices.png', dpi=150, bbox_inches='tight')
plt.show()
```



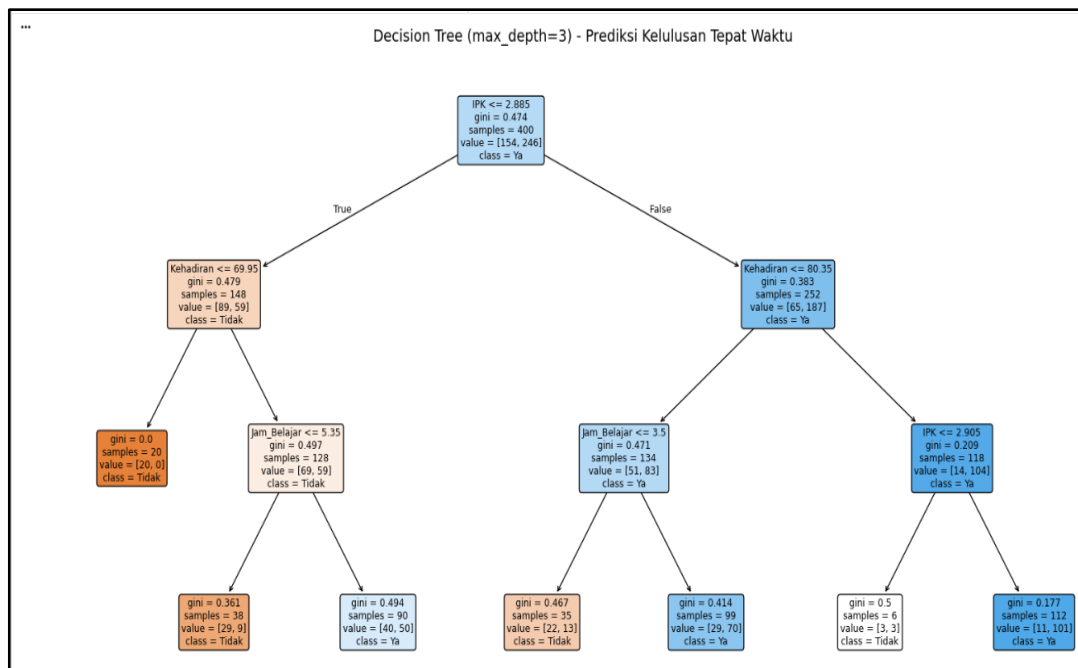
Analisis:

Confusion matrix ketiga model menunjukkan pola yang konsisten: seluruh model lebih baik dalam mengklasifikasikan kelas "Ya" dibanding kelas "Tidak", yang tercermin dari classification report sebelumnya (recall kelas "Tidak" hanya 0.36 untuk depth=3, 0.51 untuk depth=None, dan 0.54 untuk Naive Bayes — jauh lebih rendah dibanding recall kelas "Ya" yang di atas 0.74 untuk semua model). Pola ini kemungkinan besar dipengaruhi oleh distribusi kelas pada data training yang tidak sepenuhnya seimbang (61% Ya vs 39% Tidak), sehingga model cenderung "belajar lebih banyak" tentang pola kelas mayoritas dan kurang sensitif terhadap kelas minoritas. Ini menjadi catatan penting: meski akurasi keseluruhan terlihat cukup baik (terutama Naive Bayes di 0.76), performa model dalam mendeteksi mahasiswa yang berisiko tidak lulus tepat waktu (kelas "Tidak") masih perlu ditingkatkan, misalnya dengan teknik penyeimbangan data seperti oversampling atau pemberian bobot kelas (class weighting).

4. Visualisasi

4A. Plot Pohon Keputusan (max_depth=3)

```
plt.figure(figsize=(20, 10))
plot_tree(dt3, feature_names=feature_cols, class_names=target_le.classes_,
          filled=True, rounded=True, fontsize=9)
plt.title("Decision Tree (max_depth=3) - Prediksi Kelulusan Tepat Waktu", fontsize=14)
plt.savefig('decision_tree_plot.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analisis:

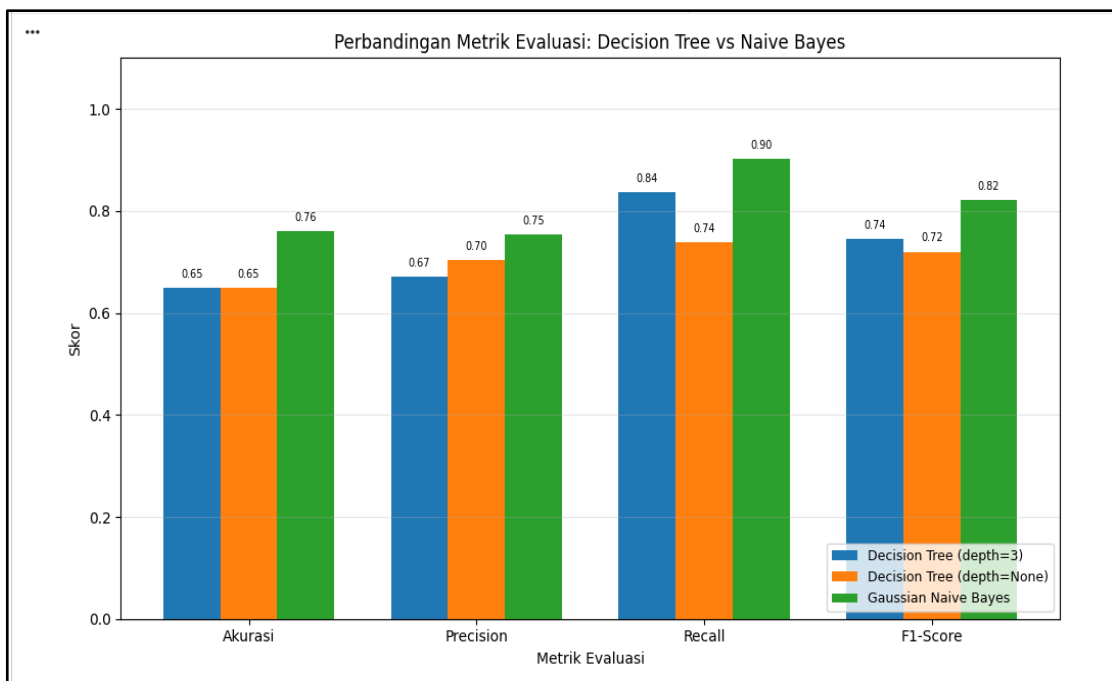
Visualisasi pohon menunjukkan split pertama dan paling signifikan terjadi pada fitur IPK dengan ambang ≤ 2.885 , memisahkan 400 data training menjadi cabang kiri (148 data, mayoritas "Tidak") dan cabang kanan (252 data, mayoritas "Ya"). Pada level kedua, kedua cabang sama-sama displit lagi berdasarkan Kehadiran (ambang 69.95 di cabang kiri, 80.35 di cabang kanan), dan pada level ketiga menggunakan Jam_Belajar atau IPK kembali. Urutan split ini mengonfirmasi bahwa IPK adalah prediktor terkuat, diikuti Kehadiran, lalu Jam_Belajar — sementara fitur kategorikal seperti Organisasi, Penghasilan_Ortu, Jenis_Kelamin, dan Status_Basiswa sama sekali tidak muncul dalam pohon dangkal ini, mengindikasikan kontribusinya jauh lebih kecil dibanding tiga fitur numerik akademik. Temuan ini relevan untuk Bagian C.2 karena menunjukkan bahwa model secara eksplisit tidak mengandalkan fitur sensitif seperti Penghasilan_Ortu atau Jenis_Kelamin untuk membuat keputusan pada level pohon dangkal ini.

4B. Bar Chart Perbandingan Metrik

```
metrics = ['Akurasi', 'Precision', 'Recall', 'F1-Score']
x = np.arange(len(metrics))
width = 0.25

fig, ax = plt.subplots(figsize=(10, 6))
for i, (name, _) in enumerate(models.items()):
    values = results_df[results_df['Model'] == name][metrics].values.flatten()
    ax.bar(x + i * width, values, width, label=name)
    for j, v in enumerate(values):
        ax.text(x[j] + i * width, v + 0.02, f"{v:.2f}", ha='center', fontsize=7)

ax.set_xlabel('Metrik Evaluasi')
ax.set_ylabel('Skor')
ax.set_title('Perbandingan Metrik Evaluasi: Decision Tree vs Naive Bayes')
ax.set_xticks(x + width)
ax.set_xticklabels(metrics)
ax.set_ylim(0, 1.1)
ax.legend(loc='lower right', fontsize=9)
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('perbandingan_metrik.png', dpi=150, bbox_inches='tight')
plt.show()
```



Analisis:

Bar chart secara visual mempertegas temuan pada poin 3a: batang hijau (Naive Bayes) konsisten lebih tinggi dibanding dua batang Decision Tree pada keempat metrik, dengan selisih paling mencolok terlihat pada Recall (0.90 berbanding 0.84 dan 0.74).

Visualisasi ini juga memperlihatkan bahwa kedua Decision Tree memiliki tinggi batang Akurasi yang identik (0.65), namun sedikit berbeda pada Precision dan Recall — pola naik-turun berlawanan arah ini adalah representasi visual dari trade-off precision-recall yang terjadi akibat perbedaan kompleksitas (kedalaman) pohon. Secara keseluruhan, chart ini memberikan bukti visual yang mendukung rekomendasi model pada Bagian C.1, di mana Naive Bayes tampak sebagai pilihan yang lebih unggul secara kuantitatif untuk kasus prediksi ini.

5. Analisis Error

```
X_test_reset = X_test.reset_index(drop=True)
y_test_reset = y_test.reset_index(drop=True)
y_pred_series = pd.Series(y_pred_dt3, name='Prediksi')

error_df = X_test_reset.copy()
error_df['Aktual'] = target_le.inverse_transform(y_test_reset)
error_df['Prediksi'] = target_le.inverse_transform(y_pred_series)
error_df['IPK'] = error_df['IPK'].round(2)
error_df['Kehadiran'] = error_df['Kehadiran'].round(1)
error_df['Jam_Belajar'] = error_df['Jam_Belajar'].round(1)

misclassified = error_df[error_df['Aktual'] != error_df['Prediksi']]
print(f"Jumlah data salah prediksi: {len(misclassified)} dari {len(error_df)} data test")

sample_errors = misclassified.head(5)
sample_errors[['IPK', 'Kehadiran', 'Jam_Belajar', 'Organisasi', 'Aktual', 'Prediksi']]
```

*** Jumlah data salah prediksi: 35 dari 100 data test

	IPK	Kehadiran	Jam_Belajar	Organisasi	Aktual	Prediksi
4	2.89	57.3	6.0	0	Tidak	Ya
6	2.04	74.1	12.8	1	Tidak	Ya
8	2.63	98.6	4.6	1	Ya	Tidak
11	1.54	100.0	9.7	1	Tidak	Ya
12	2.44	87.3	9.0	0	Tidak	Ya

Analisis:

Dari 100 data uji, model Decision Tree (max_depth=3) salah memprediksi 35 data (error rate 35%). Pola pada lima sampel yang diperiksa menunjukkan karakteristik yang konsisten: sebagian besar kesalahan terjadi pada mahasiswa dengan nilai fitur yang berada tepat di sekitar ambang split pohon, misalnya IPK di kisaran 2.0-2.9 (dekat ambang split pertama 2.885) atau Kehadiran di kisaran 57-98% yang melewati berbagai ambang split (69.95 dan 80.35). Karena Decision Tree membuat keputusan biner yang tegas pada setiap node (bukan probabilistik), mahasiswa dengan kondisi "borderline" semacam ini sangat rentan salah klasifikasi meski secara substansi kondisinya

sebenarnya ambigu, misalnya satu kasus dengan IPK 2.63 dan Kehadiran sangat tinggi (98.6%) tetap diprediksi "Tidak" karena split IPK ≤ 2.905 pada cabang kanan pohon tidak lagi mempertimbangkan Kehadiran yang sebenarnya sangat mendukung. Pembatasan max_depth=3 turut memperbesar risiko ini karena pohon tidak sempat mempertimbangkan fitur tambahan (seperti Jam_Belajar atau Organisasi) untuk kasus-kasus yang sebenarnya butuh informasi lebih dari tiga level split yang tersedia, ini adalah bukti nyata dari trade-off klasik antara interpretabilitas (pohon dangkal, mudah dibaca) dan akurasi (pohon dalam, lebih akurat tapi berisiko overfitting) yang telah dibahas secara konseptual di Bagian A no. 3.

BAGIAN C: REKOMENDASI & ETIKA [30%] – Kelompok

1. Rekomendasi Model: Dari 3 model, mana yang kamu pilih untuk dipakai kampus? Jelaskan dengan data dari Bagian B

Jawab:

Dari ketiga model yang diuji, Gaussian Naive Bayes dipilih sebagai model yang paling layak dipakai kampus. Hasil evaluasi pada 100 data testing di Bagian B menunjukkan Naive Bayes memperoleh akurasi 0.76, precision 0.75, recall 0.90, dan F1-score 0.82, yang seluruhnya lebih tinggi dibanding Decision Tree depth=3 (akurasi 0.65, F1 0.74) maupun depth=None (akurasi 0.65, F1 0.72). Yang membuat Naive Bayes lebih meyakinkan sebagai pilihan bukan sekadar unggul di satu metrik, melainkan unggul konsisten di keempat metrik sekaligus, berbeda dengan dua varian Decision Tree yang justru saling bertukar posisi antara precision dan recall (trade-off klasik akibat perbedaan kedalaman pohon). Recall 0.90 pada Naive Bayes juga punya arti praktis yang penting bagi kampus: nilai ini menunjukkan model jarang meleset mengenali mahasiswa yang sebenarnya akan lulus tepat waktu, sehingga risiko salah klasifikasi (baik untuk kelas "Ya" maupun "Tidak") bisa ditekan lebih baik dibanding kedua Decision Tree. Ditambah lagi, kedua varian Decision Tree sama-sama mentok di akurasi 0.65 walau kompleksitasnya jauh berbeda, mengindikasikan bahwa menambah kedalaman pohon di dataset ini tidak memberi manfaat generalisasi apa pun, hanya menambah risiko overfitting. Dari segi efisiensi komputasi, Naive Bayes juga lebih ringan karena tidak perlu mencari kombinasi split terbaik seperti Decision Tree, sehingga lebih praktis dijalankan ulang tiap semester dengan data mahasiswa yang terus bertambah.

2. Bias & Etika: Fitur "Penghasilan_Ortu" dan "Jenis_Kelamin" rawan bias. Apa dampaknya jika model dipakai untuk beasiswa? Bagaimana mitigasinya?

Jawab:

Fitur Penghasilan_Ortu dan Jenis_Kelamin berpotensi menimbulkan masalah serius jika model ini dipakai untuk menentukan penerima beasiswa. Sebagaimana sudah disinggung di Bagian A, Penghasilan_Ortu kemungkinan besar berkorelasi kuat dengan Status_Beasiswa karena secara historis penerima beasiswa memang ditentukan dari penghasilan orang tua. Kalau model dilatih dari data semacam ini lalu dipakai lagi untuk memutuskan siapa yang berhak dapat beasiswa, yang terjadi sebenarnya bukan penilaian objektif atas kelayakan mahasiswa, melainkan pengulangan pola keputusan lama secara otomatis. Ditambah lagi, proses encoding yang dipakai (Menengah=0, Rendah=1, Tinggi=2) memberi urutan angka pada kategori yang sebenarnya tidak punya tingkatan matematis, sehingga ada risiko model "membaca" penghasilan tinggi sebagai nilai yang secara numerik lebih besar padahal itu bukan makna sebenarnya. Untuk Jenis_Kelamin, kalau fitur ini ikut memengaruhi keputusan model, bisa saja satu gender diuntungkan atau dirugikan dalam seleksi beasiswa tanpa alasan yang berkaitan dengan prestasi akademik, dan ini jelas bentuk perlakuan yang tidak adil sekaligus berpotensi melanggar prinsip non-diskriminasi yang seharusnya dipegang institusi pendidikan. Sebagai langkah mitigasi, cara paling sederhana dan langsung adalah mengeluarkan kedua fitur ini dari input model, terutama kalau modelnya dipakai untuk keputusan beasiswa, bukan sekadar prediksi kelulusan biasa. Menariknya, temuan di visualisasi pohon pada poin 4A justru sudah mendukung langkah ini secara alami, karena pada Decision Tree yang dangkal, Penghasilan_Ortu dan Jenis_Kelamin sama sekali tidak muncul sebagai fitur pemisah, kalah jauh kontribusi prediktifnya dibanding IPK, Kehadiran, dan Jam_Belajar. Selain menghapus fitur sensitif, performa model juga sebaiknya dievaluasi terpisah per kelompok, misalnya membandingkan recall antara laki-laki dan perempuan atau antar kategori penghasilan, untuk memastikan tidak ada kelompok tertentu yang secara sistematis dirugikan oleh prediksi model. Terakhir, hasil model idealnya tetap diposisikan sebagai bahan pertimbangan, bukan keputusan final otomatis, sehingga tetap ada proses tinjauan manusia sebelum keputusan beasiswa benar-benar diambil.

3. Simulasi: Jika 1 mahasiswa punya IPK tinggi tapi Kehadiran rendah, prediksi kedua model apa? Kenapa bisa beda?

Jawab:

Untuk mahasiswa dengan IPK tinggi tapi Kehadiran rendah, kedua model berpotensi memberi hasil yang berbeda, dan ini bisa dijelaskan langsung dari struktur pohon di Bagian

B.4A. Split pertama Decision Tree terjadi pada $IPK \leq 2.885$, sehingga mahasiswa dengan IPK tinggi akan masuk ke cabang kanan yang mayoritas berlabel "Ya". Namun split berikutnya di cabang itu adalah Kehadiran ≤ 80.35 , artinya begitu Kehadiran-nya rendah, mahasiswa tadi turun ke sub-cabang yang labelnya bisa berbalik condong ke "Tidak", tergantung split level ketiga yang menggunakan Jam_Belajar atau IPK lagi. Pola ini persis seperti temuan di analisis error Bagian B.5, di mana mahasiswa dengan IPK 2.63 dan Kehadiran sangat tinggi (98.6%) tetap diprediksi "Tidak" karena jalur split yang dilaluinya kebetulan tidak lagi mempertimbangkan Kehadiran. Jadi Decision Tree cenderung sangat bergantung pada jalur atau urutan split yang dilewati satu kasus, bukan hanya pada nilai masing-masing fitur secara terpisah.

Naive Bayes bekerja dengan logika berbeda. Model ini menghitung seberapa besar IPK tinggi mendukung kelas "Ya" dan seberapa besar Kehadiran rendah mendukung kelas "Tidak" secara terpisah, lalu mengalikan kedua bukti tersebut untuk mendapat probabilitas akhir. Karena IPK sudah terbukti sebagai prediktor paling kuat di dataset ini (jadi split pertama dengan pemisahan kelas paling tajam), sinyal dari IPK tinggi kemungkinan besar tetap mendominasi hasil akhir meski ada sinyal berlawanan dari Kehadiran rendah, sehingga Naive Bayes berpeluang tetap memprediksi "Ya".

Perbedaan hasil ini pada dasarnya adalah konsekuensi langsung dari perbedaan asumsi yang sudah dibahas di Bagian A nomor 1: Decision Tree menangkap hubungan bersyarat antar fitur, semacam aturan "kalau IPK tinggi dan Kehadiran-nya juga rendah, maka...", sedangkan Naive Bayes mengasumsikan setiap fitur berkontribusi secara independen tanpa memperhitungkan kombinasi keduanya secara khusus. Untuk kasus borderline seperti IPK tinggi berbanding Kehadiran rendah ini, Decision Tree berpotensi lebih peka menangkap risiko nyata dari kombinasi tersebut, sementara Naive Bayes bisa jadi terlalu optimis karena kekuatan sinyal IPK yang tinggi "menutupi" sinyal negatif dari Kehadiran yang rendah.